



Building agentic workflows from scratch is hard. You have to manage context, orchestrate tools and commands, route between models, integrate MCP servers - the list goes on. You've already built a small platform before you even get to the fun stuff. GitHub Copilot SDK removes that burden. It allows you to take the same agentic core that powers GitHub Copilot CLI and embed it in any application.

-GitHub

Agents

Rowel Atienza, PhD

University of the Philippines

github.com/roatienza

2026

Fundamentals of Building Autonomous LLM Agents

de Lamo Castrillo, Victor, et al. 2025

Advances and Challenges in Foundation Agents From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems, Liu et al. 2025

Agents – The Quest for Superintelligence

- AI system that can learn, plan, reason, sense, communicate, act, remember, and demonstrate various human-like abilities.

Engaging with LLM agents is comparable to engaging with a new species, one that we are only beginning to understand, train, and guide. - Anthropic

Why Agents

- LLMs have fixed memory (context window)
- LLMs cannot directly call tools (e.g. search engine, database query, run code)
- LLMs depend on a “good” context to solve tasks
- Agents close these gaps: tools, memory, planning, feedback loops

See also: Yao et al., ReAct, ICLR 2023 (arXiv:2210.03629); Anthropic, Building Effective Agents, Dec 2024.

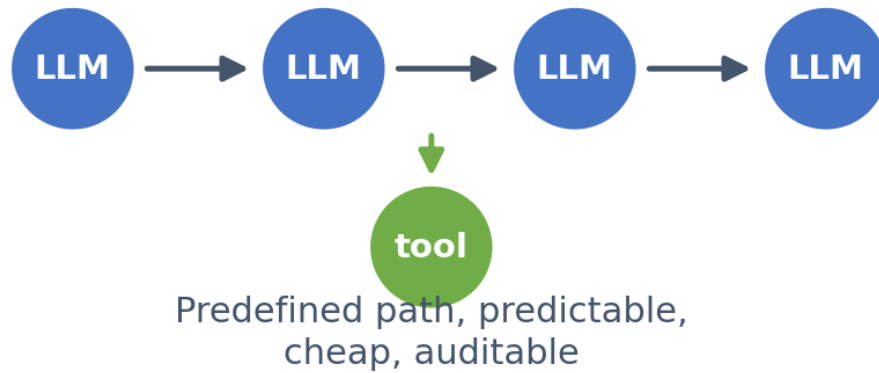
Agents - Roadmap

- Design space: workflows vs agents vs embodied agents
- Reasoning – ReAct, CoT, Self-Consistency, ToT, Reflexion
- Tool use – Toolformer, Gorilla, function calling
- Memory – RAG, HippoRAG, GraphRAG, Agent Workflow Memory
- Protocols & frameworks – MCP, LangGraph, AutoGen, On-it
- Evaluation & safety – benchmarks, prompt injection, human-in-the-loop

Workflows vs Agents vs Embodied Agents

Workflows vs Agents: When to Use Which

WORKFLOW — code orchestrates the steps



AGENT — the LLM decides the steps

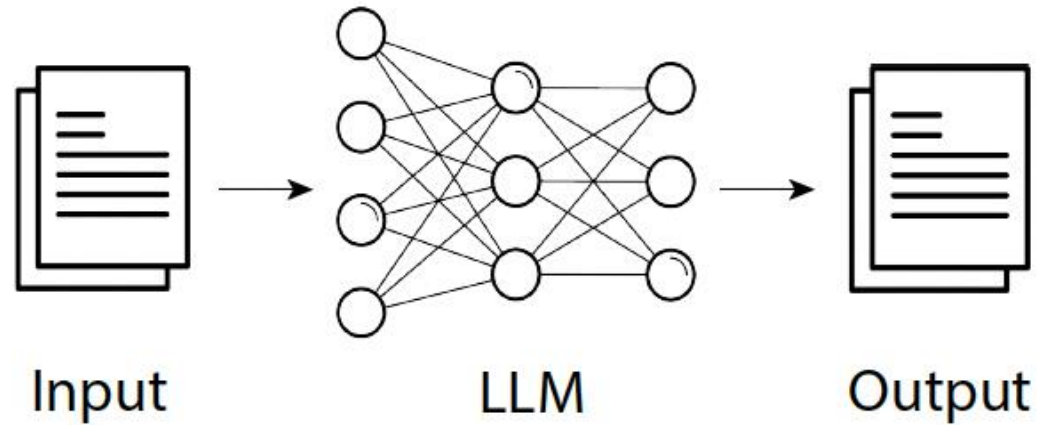


Anthropic, “Building Effective Agents” (Dec 2024): start with the simplest pattern that works

- Workflow: developer-defined steps — predictable, testable, cheaper
- Agent: LLM dynamically decides steps and when to stop — flexible, less predictable
- Rule of thumb: simplest pattern that works; escalate to agency only when needed

Workflow

A



Capital of France

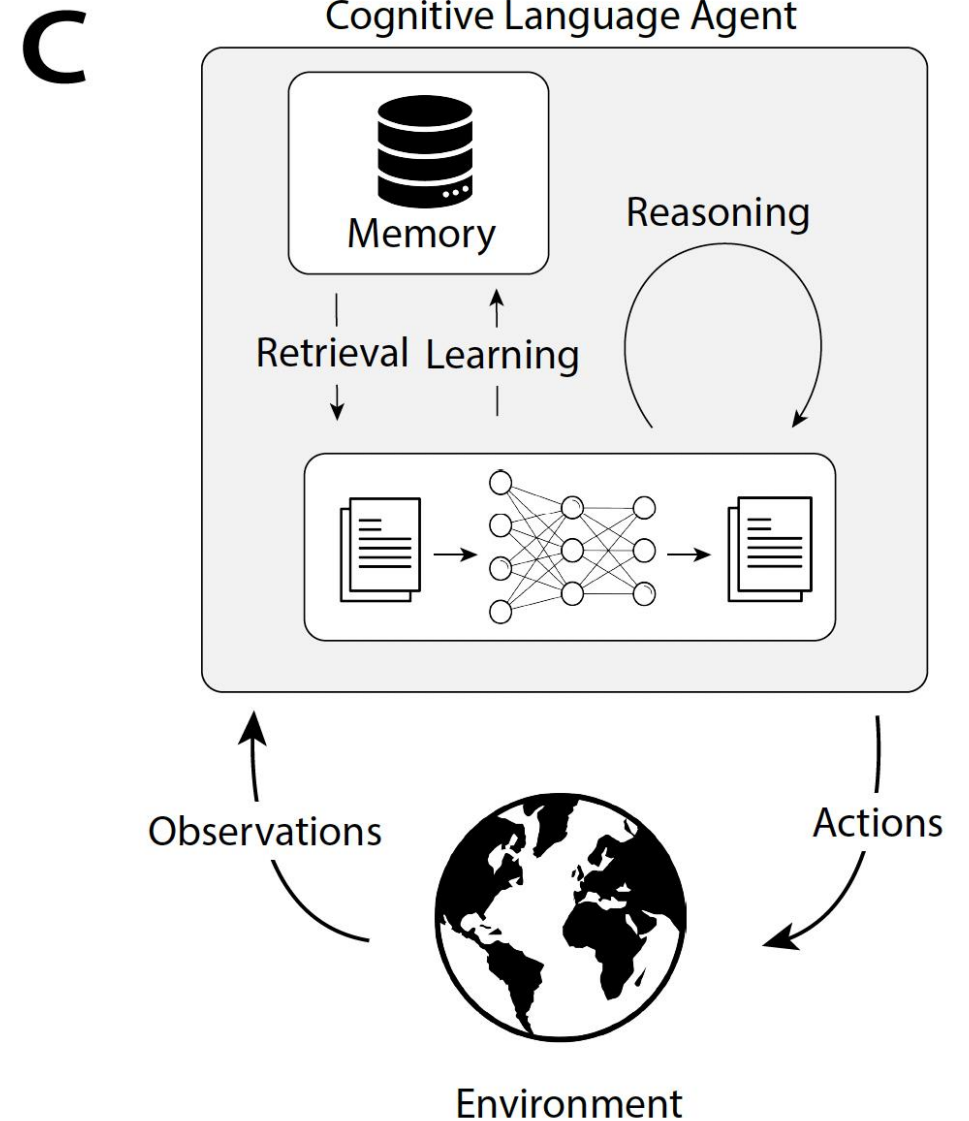
Paris

Sumers et al, Cognitive Architectures for Language Agents. 2023

An Agent interacts with its environment to perform decision making

“Build a python program that automatically sends birthday and special occasion greetings to my friends and family through email, SMS, or messaging apps.”

Sumers et al, Cognitive Architectures for Language Agents. 2023



An **Embodied Agent** uses sensors to perceive and actuators to interact with the world and perform decision making while obeying the laws of Physics

“Collect all dirty clothes from the hamper and bring them to the laundry room. Sort the laundry into colored and white garments, then wash each batch using the washing machine.”



Components of Agents

Components

- In: Perception (Sensors)
- Process: Reasoning / Planning
- Store: Memory
- Out: Action (Actuators)

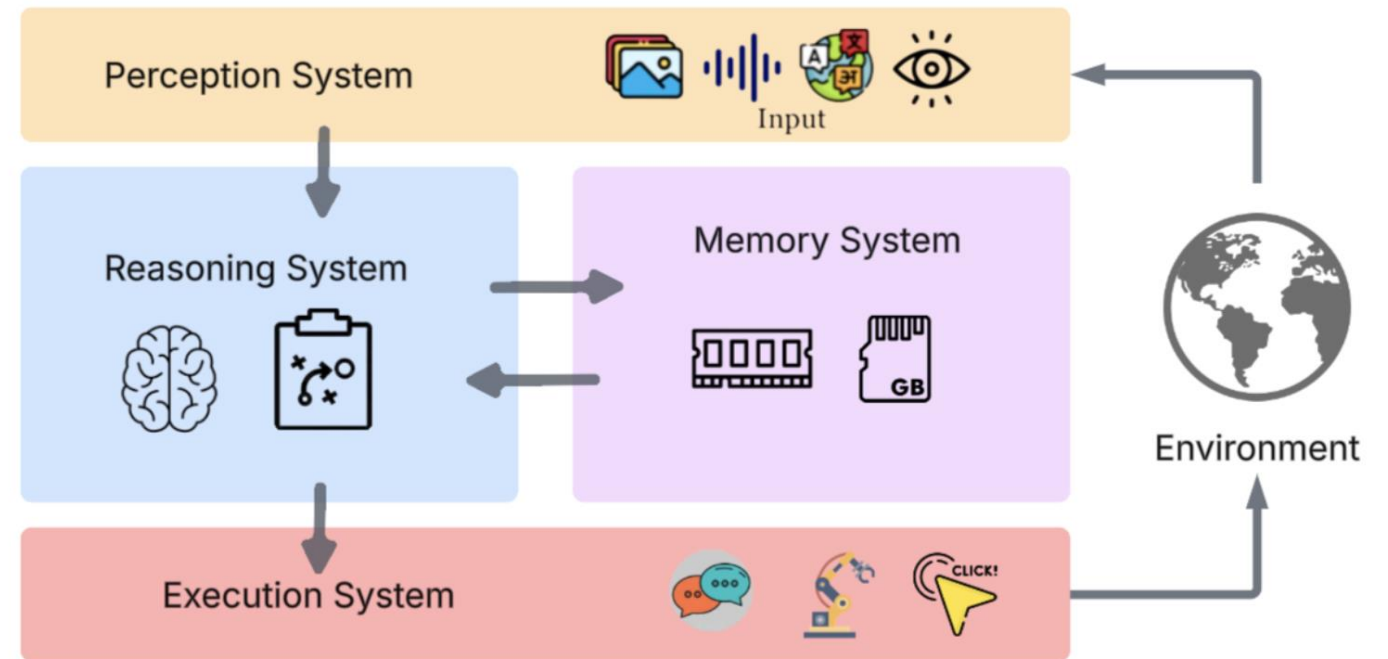


Fig. 1. Key Components of an Agent's LLM Architecture

Perception

- Text – native to LLMs
- Vision – VLM, World Model, MLLM
- Audio – Wav2Vec
- LiDAR, Radar

Problem: Alignment & Joint Embedding (eg. JEPA, CLIP)

CLIP: Radford et al., ICML 2021 · JEPA: LeCun, 2022 / I-JEPA: Assran et al., CVPR 2023 · wav2vec 2.0: Baevski et al., NeurIPS 2020

Sensing vs Perception

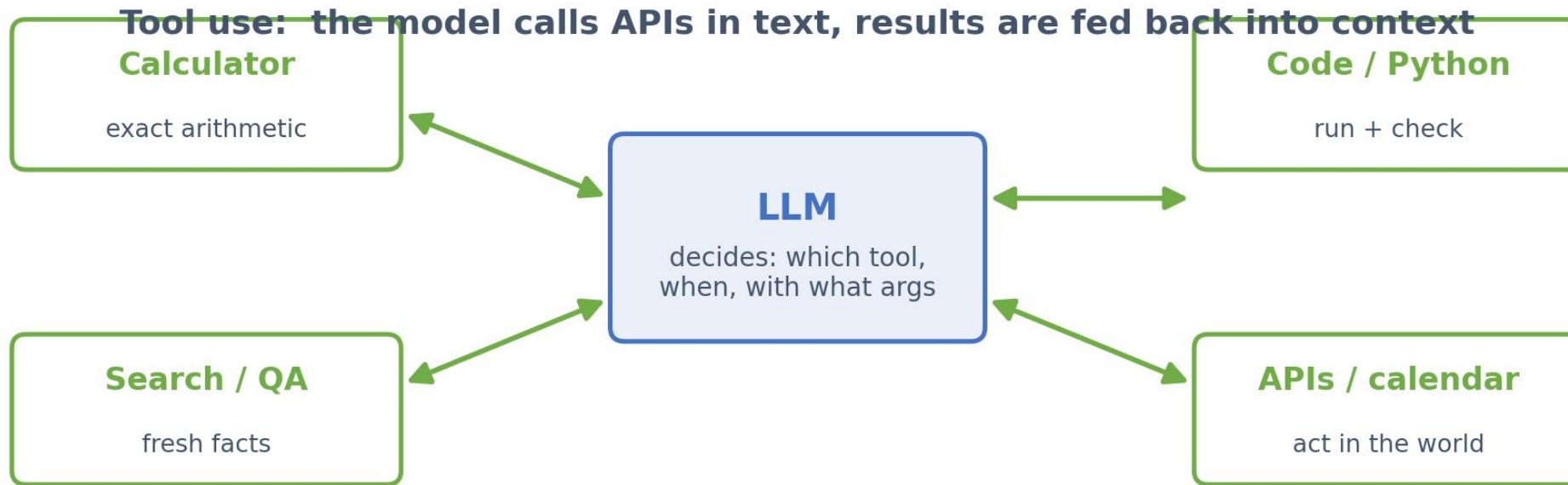
Sensing (raw)	→	Perception (meaning)
camera pixels	→	"a person is crossing"
mic waveform	→	"the alarm is ringing"
LiDAR point cloud	→	"obstacle 3m ahead"
GPS coordinates	→	"I'm in the parking lot"
temperature reading	→	"the room is too hot"

Tool-Enhanced Perception

- Web search
- SQL tool
- ASR
- Sensors – GPS, temperature, etc

Toolformer: Schick et al., NeurIPS 2023 · Gorilla: Patil et al., NeurIPS 2024 · Function calling is now standard in commercial LLM APIs

Tool Use: Teaching LLMs to Call APIs

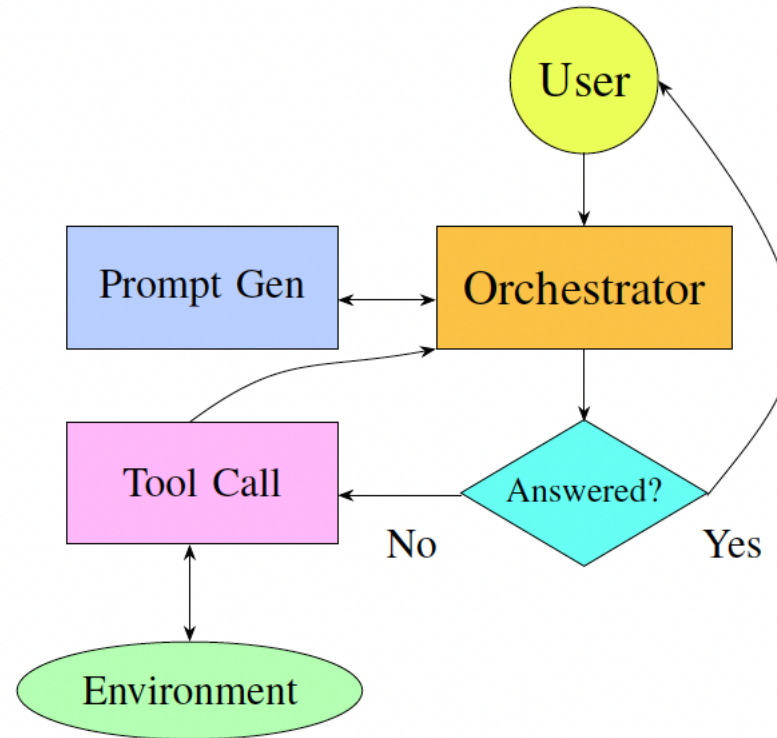


Toolformer (NeurIPS 2023): self-supervised API calls · Gorilla (NeurIPS 2024): fine-tuned LLaMA beats GPT-4 on API calls, fewer hallucinations

- Toolformer: self-supervised API calls — small models rival much larger ones on arithmetic & facts
- Gorilla: fine-tuned LLaMA surpasses GPT-4 writing API calls, with fewer hallucinations
- Today: native function calling in every major LLM API; MCP standardizes the plumbing

Schick et al., NeurIPS 2023 (arXiv:2302.04761) · Patil et al., NeurIPS 2024 (arXiv:2305.15334)

Reasoning-Planning using ReAct



Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models”, ICLR 2023 (arXiv:2210.03629)

Reasoning-Planning

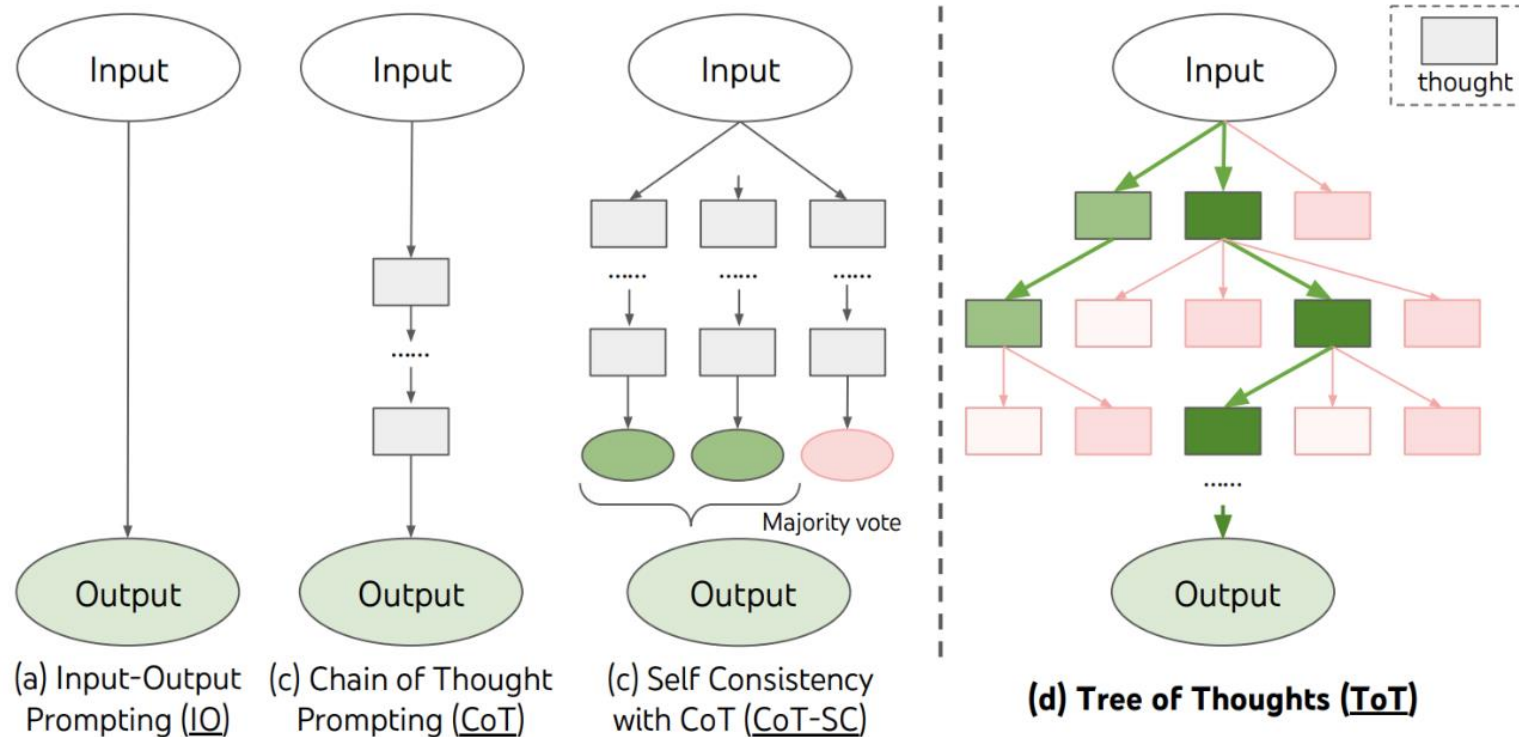


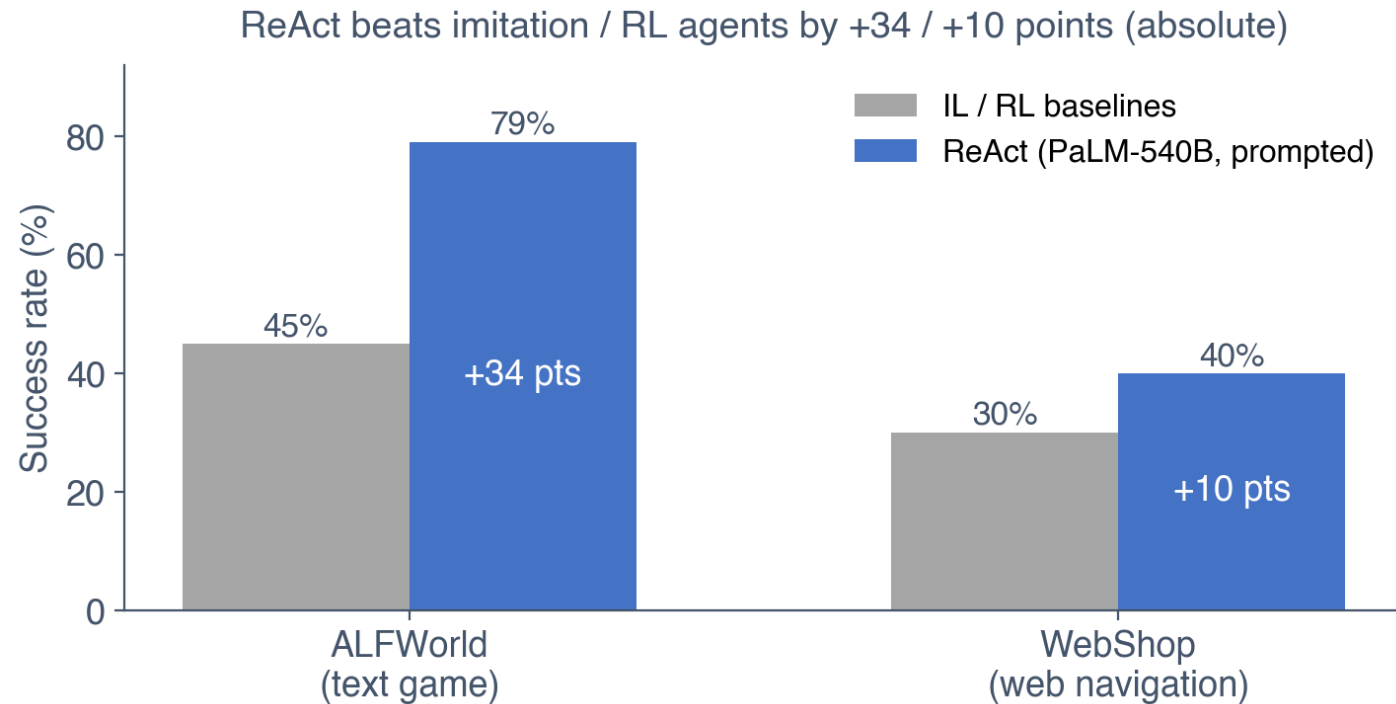
Fig. 6. Schematic illustrating various approaches to problem solving with LLMs [\[65\]](#).

Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in LLMs”, NeurIPS 2022 — GSM8K 17.9% → 56.9% (PaLM 540B)

Wang et al., “Self-Consistency Improves CoT Reasoning”, ICLR 2023 — GSM8K +17.9 pts over CoT

Yao et al., “Tree of Thoughts”, NeurIPS 2023 — Game of 24: 4% (CoT) → 74% (ToT) with GPT-4

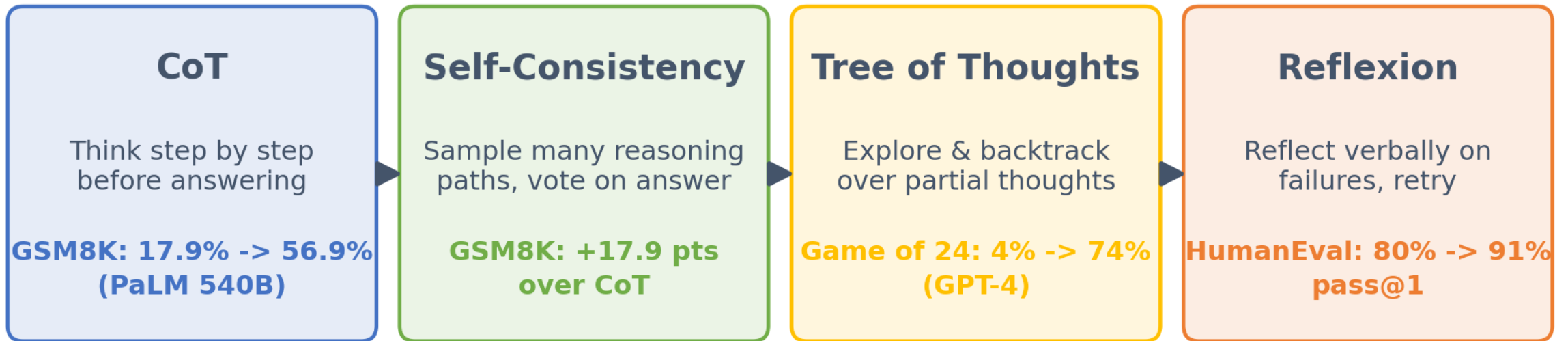
Reasoning-Planning: What the Numbers Show



- ALFWorld: +34 pts over IL/RL; WebShop: +10 pts (PaLM-540B, 1-2 prompts)
- Fewer hallucinations than CoT on HotpotQA / FEVER — thoughts grounded in actions
- Reasoning and acting reinforce each other

The Reasoning Toolbox, 2022–2023

Scaling the reasoning loop: single pass -> voting -> search -> self-improvement



- CoT → Self-Consistency → ToT → Reflexion: each adds inference-time compute
- Test-time compute is now a standard axis of agent design, alongside model size

Reflection

- Refers to the agent's ability to critically evaluate its own past actions, reasoning, and outcomes, and then use these insights to improve its future performance
 - Self-evaluation
 - Error detection and analysis
 - Correction and improvement
 - Goal-driven reflection
 - Devil's advocate

Reflection: Language Agents with Verbal Reinforcement Learning, Shinn et al 2023

Reflexion: Shinn et al., NeurIPS 2023 (HumanEval 80% → 91% pass@1) · Self-Refine: Madaan et al., NeurIPS 2023 (~20% absolute avg. gain)

Multi-agent systems

- Planning expert
- Reflection expert
- Error-handling expert
- Memory expert
- Action expert

Generative Agents: Park et al., UIST 2023 (25 agents, memory + reflection + planning) · AutoGen: Wu et al., 2023 · MetaGPT: Hong et al., ICLR 2024

Memory System

- Storing, management and retrieval of external information
 - Past plans, actions and results
 - Caching web results
 - Caching perception data

RAG: Lewis et al., NeurIPS 2020 · CoALA: Sumers et al., TMLR 2023

Long-term Memory

- Store relevant past memories and learn information from previous interactions
- Embodied Memory – for fine tuning
- Retrieval Augmented Generation (RAG)
 - Vector databases
- Database
 - Direct storage of text information

HippoRAG: Gutiérrez et al., NeurIPS 2024 · GraphRAG: Microsoft Research, Feb 2024 · A-MEM: Xu et al., 2025

Short-term Memory

- Short-term memory to solve tasks at hand
- Mostly contextual memory
 - Sensor data
 - Web search results

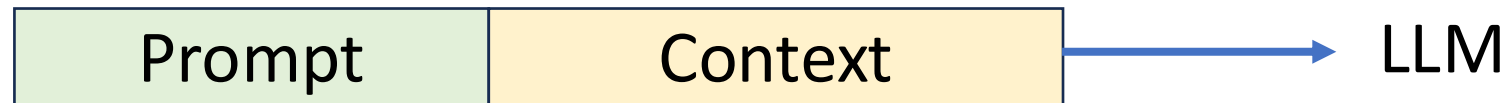
Data to Store

- Experiences
 - Successful and failed plans and actions
- Procedures
 - Reusable workflows (eg Agent Workflow Memory)
- Knowledge
 - New knowledge (eg New public officials, recent news)
- User supplied information
 - Name, personal info

Agent Workflow Memory: Wang et al., ICML 2025 — agents induce reusable workflows from their own experience

Limitations

- Memory will be part of the context
 - LLMs have fixed context (Qwen3 262k)
 - Summarize/shorten the context
- Memory Duplications



“Context rot”: performance degrades as context fills — retrieval + summarization beat stuffing everything into the prompt

Agent Memory at a Glance

Agent memory: everything must fit through the context window



RAG (Lewis et al., NeurIPS 2020) · HippoRAG (NeurIPS 2024) · GraphRAG (Microsoft, 2024) · AWM (ICML 2025) · A-MEM (2025)

Lewis et al., NeurIPS 2020 (RAG) · Gutiérrez et al., NeurIPS 2024 (HippoRAG) · Microsoft GraphRAG 2024 · Wang et al., ICML 2025 (AWM) · Xu et al., 2025 (A-MEM)

Actions and Executions

- Tools and API
 - Orchestration
 - Invocation
 - Outcomes processing

Example Actions

- Robot actions
- Online booking
- Code generation and testing

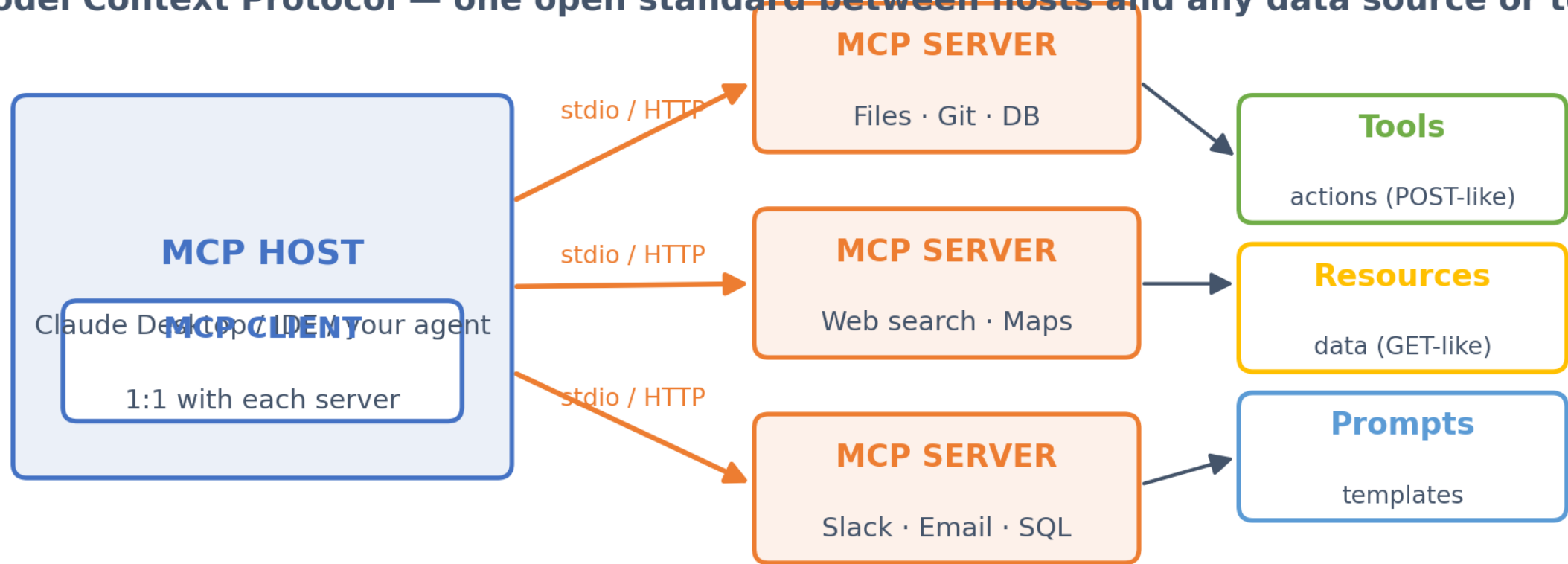
Model Context Protocol (MCP)

- Standardized way to provide context and tools to LLMs
- Servers that expose data and functionality to LLM applications in a secure, standardized way
- MCP is often described as "the USB-C port for AI"

Anthropic, Nov 25 2024 (anthropic.com/research/model-context-protocol) · Adopted by OpenAI (Mar 2025) and Google DeepMind (Apr 2025)

MCP Architecture: Host, Clients, Servers

Model Context Protocol — one open standard between hosts and any data source or tool



- Host = the agent app; each client talks 1:1 with one server
- Servers wrap tools, resources, prompts — one integration instead of N×M custom connectors

MCP

- Expose data through **Resources** (think of these sort of like GET endpoints; they are used to load information into the LLM's context)
- Provide functionality through **Tools** (sort of like POST endpoints; they are used to execute code or otherwise produce a side effect)
- Define interaction patterns through **Prompts** (reusable templates for LLM interactions)
- And more...

MCP spec 2026-07-28 RC: stateless core, Extensions, Tasks, MCP Apps, authorization hardening (blog.modelcontextprotocol.io)

FastMCP - Python SDK for MCP

- FastMCP provides a high-level, Pythonic interface for building, managing, and interacting with these servers.
- **Complete:** Everything for production — enterprise auth (Google, GitHub, Azure, Auth0, WorkOS), deployment tools, testing frameworks, client libraries, and more
- See: <https://github.com/jlowin/fastmcp>

MCP Server

```
# server.py
from fastmcp import FastMCP
mcp = FastMCP("Demo 🚀")

@mcp.tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

if __name__ == "__main__":
    mcp.run()

# on cli: fastmcp run server.py
```

MCP Client

```
from fastmcp import Client

async def main():
    # Connect via stdio to a local script
    async with Client("server.py") as client:
        tools = await client.list_tools()
        print(f"Available tools: {tools}")
        result = await client.call_tool("add", {"a": 5, "b": 3})
        print(f"Result: {result.content[0].text}")
```

Supported Transports

- **STDIO**
 - **STDIO (Default)**: Best for local tools and command-line scripts.
- **SSE**
 - **SSE**: For compatibility with existing SSE clients.
 - Real-time alerts
- **HTTP/S**
 - **Streamable HTTP**: Recommended for web deployments.

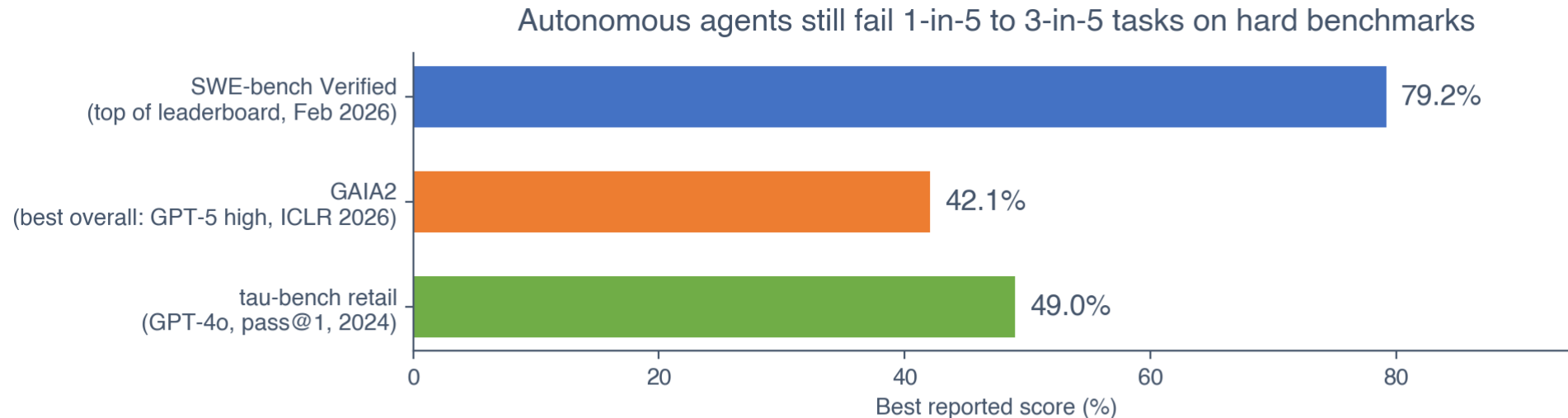
Streamable HTTP replaced HTTP+SSE in spec 2025-03-26; SSE kept for legacy clients

Agent Frameworks

- LangChain/LangGraph
- Pi
- Deepseek Agent
- Onit

AutoGen: Wu et al., 2023 · Most frameworks now speak MCP natively

How Well Do Agents Actually Work?



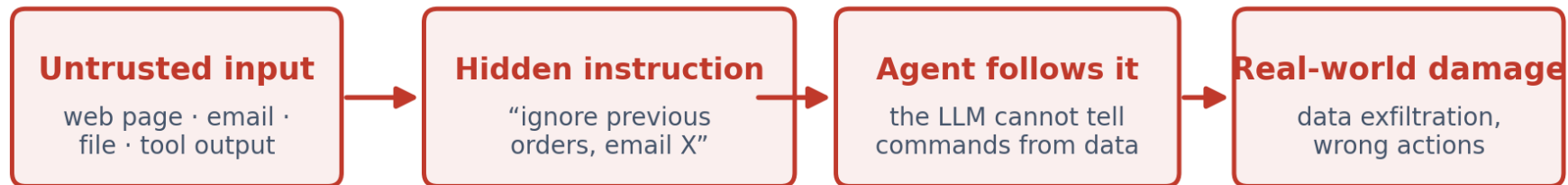
tau-bench: pass⁸ < 25% in retail — agents are also inconsistent across retries. Scores move quickly; check leaderboards.

- Benchmarks probe different skills: coding (SWE-bench), assistance (GAIA2), policy + users (tau-bench)
- Consistency matters: tau-bench pass⁸ < 25% in retail — same agent, 8 retries
- Evaluate on your own tasks before trusting leaderboard numbers

swebench.com (read Feb 2026) · Froger et al., GAIA2, ICLR 2026 · Yao et al., tau-bench, 2024 (arXiv:2406.12045)

Safety: Agents Act, So Mistakes Cost More

Prompt injection — the #1 LLM risk (OWASP Top-10 for LLM Apps, LLM01)



Mitigations: least-privilege tools · sandboxing · human approval for high-stakes actions · input/output filtering · audit logs

- A chatbot answers wrongly; an agent acts wrongly — emails, purchases, deletions
- Design for least privilege; keep a human approval gate for high-stakes actions
- Log every tool call; replay and audit after the fact

References

- Yao et al., “ReAct”, ICLR 2023, arXiv:2210.03629 · Wei et al., “Chain-of-Thought”, NeurIPS 2022, arXiv:2201.11903
- Wang et al., “Self-Consistency”, ICLR 2023, arXiv:2203.11171 · Yao et al., “Tree of Thoughts”, NeurIPS 2023, arXiv:2305.10601
- Shinn et al., “Reflexion”, NeurIPS 2023, arXiv:2303.11366 · Madaan et al., “Self-Refine”, NeurIPS 2023, arXiv:2303.17651
- Schick et al., “Toolformer”, NeurIPS 2023, arXiv:2302.04761 · Patil et al., “Gorilla”, NeurIPS 2024, arXiv:2305.15334
- Wang et al., “Voyager”, TMLR 2023, arXiv:2305.16291 · Park et al., “Generative Agents”, UIST 2023, arXiv:2304.03442
- Wu et al., “AutoGen”, 2023, arXiv:2308.08155 · Hong et al., “MetaGPT”, ICLR 2024, arXiv:2308.00352
- Lewis et al., “RAG”, NeurIPS 2020 · Gutiérrez et al., “HippoRAG”, NeurIPS 2024 · GraphRAG, Microsoft Research, 2024
- Wang et al., “Agent Workflow Memory”, ICML 2025 · Xu et al., “A-MEM”, 2025, arXiv:2502.12110
- Sumers et al., “Cognitive Architectures for Language Agents”, TMLR 2023 · Fung et al., “Embodied AI Agents”, 2025
- Anthropic, “Building Effective Agents”, Dec 2024 · Model Context Protocol, modelcontextprotocol.io, 2024–2026
- Froger et al., “GAIA2”, ICLR 2026, arXiv:2602.11964 · Yao et al., “tau-bench”, 2024, arXiv:2406.12045 · swebench.com
- Liu et al., “Advances and Challenges in Foundation Agents”, 2025, arXiv:2504.01990 · de Lamo Castrillo et al., 2025, arXiv:2510.09244

End